AprovaAí

MICROSSERVIÇO DE AUTENTICAÇÃO E AUTORIZAÇÃO

Especificação técnica concisa de arquitetura e regras de segurança

JWT | RBAC | PROTEÇÃO DE CURSOS

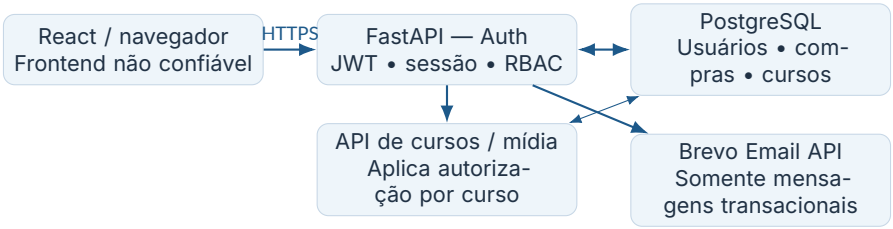
ALUNO • PROFESSOR • CONVIDADO • ADMIN

FastAPI • React • PostgreSQL • Brevo API • Swagger UI

Documento	AprovaAi-SEC-001
Versão	1.3 — políticas de senha e acesso admin atualizadas
Data	23/10/2026
Status	Proposta de implementação; regras comerciais e operacionais sujeitas à validação

1 Escopo e arquitetura do microsserviço

O **Microsserviço de Autenticação e Autorização do AprovaAi** é responsável pelo cadastro, confirmação de email, autenticação, emissão e renovação de tokens, sessões, atribuição de papéis e políticas de acesso. A aplicação utilizará **JWT para usuários autenticados**; **CONVIDADO não se autentica e não recebe JWT**. **RBAC** é o modelo principal, com checagem adicional do vínculo entre pessoa e curso para recursos privados.



Separação de responsabilidades. O serviço de identidade autentica, administra sessões e define papéis. A API de cursos **deve verificar, a cada acesso privado**, a autorização efetiva por compra/matrícula ou autoria; não confiar em flags vindas do React. Se um serviço consumir JWT de outro, deve verificar assinatura, emissor, destinatário e validade.

2 Decisões técnicas adotadas

Assunto	Decisão para a versão inicial
Identidade / cadastro	UUID; email único, normalizado; senha; cadastro público cria somente **ALUNO**.
Confirmação	Email obrigatório via Brevo; conta permanece pendente até confirmar token de uso único (24 h).
Senhas	Argon2id; mínimo de 12 caracteres e aceitação de pelo menos 64 caracteres.
Autenticação	Email + senha para todos os usuários, inclusive ADMIN; login social fora do escopo inicial.
Access token	JWT assinado com EdDSA/Ed25519; expiração em **15 minutos**; validação estrita.
Refresh token	Opaco aleatório, 256 bits; **7 dias**; hash em banco; rotação e detecção de reutilização.
Frontend	Access JWT em memória; refresh em cookie `HttpOnly`, `Secure` (produção) e `SameSite` adequado.
Autorização	Negar por padrão; roles ALUNO, PROFESSOR e ADMIN; CONVIDADO = anônimo; verificação de curso.
Papéis privilegiados	PROFESSOR e ADMIN atribuídos apenas por administrador; alteração revoga sessões.
Swagger	`/docs` e `/openapi.json` somente para ADMIN autenticado e com sessão válida.
Mídia / pagamentos	Entitlement apenas após confirmação confiável de pagamento; arquivos privados nunca públicos por URL estática.

Premissas: uma role persistente por usuário, autoria primária de curso e acesso do aluno por matrícula ativa. Administração irrestrita se refere às *funcionalidades da aplicação*, não a segredos, senhas ou chaves.

3 Papéis RBAC e política de acesso

CONVIDADO: usuário anônimo, sem JWT; acesso a homepage, catálogo, descrição e conteúdos marcados como prévias públicas. **ALUNO:** usuário autenticado e ativo; somente conteúdos privados dos cursos cuja compra/matrícula lhe conceda direito de acesso. **PROFESSOR:** usuário autenticado e ativo; somente conteúdos privados dos cursos de sua autoria, além das operações de criação/edição desses cursos. **ADMIN:** conta privilegiada autenticada; todos os recursos funcionais e documentação da API.

3.1 Matriz de autorizações

Ação / recurso	Convidado	Aluno	Professor	Admin
Navegar em páginas, catálogo e preços públicos	SIM	SIM	SIM	SIM
Ver prévia pública de aula	SIM	SIM	SIM	SIM
Ler / baixar aula ou material privado	NÃO	COND.	COND.	SIM
Consultar e editar *próprio* perfil	NÃO	SIM	SIM	SIM
Consultar *próprias* compras	NÃO	SIM	NÃO	SIM
Comprar curso / iniciar checkout	NÃO	SIM	NÃO	SIM
Criar curso e aulas	NÃO	NÃO	SIM	SIM
Editar / excluir curso privado	NÃO	NÃO	COND.	SIM
Visualizar curso não publicado	NÃO	NÃO	COND.	SIM
Publicar / aprovar cursos de terceiros	NÃO	NÃO	NÃO	SIM
Administrar usuários, papéis e matrículas	NÃO	NÃO	NÃO	SIM
Consultar auditoria e área administrativa	NÃO	NÃO	NÃO	SIM
Acessar Swagger UI e esquema OpenAPI	NÃO	NÃO	NÃO	SIM

Legenda: COND. significa que o backend deve confirmar *curso específico*: direito de matrícula para ALUNO ou autoria para PROFESSOR.

Estas regras valem mesmo quando alguém descobre o identificador do recurso.

3.2 Predicado de autorização para conteúdo privado

Permitir acesso ao recurso privado do curso C se e somente se:

1. Usuário autenticado, ativo e com token/sessão válidos; **E**
2. Role = **ADMIN**; **OU** role = **ALUNO** com matrícula/entitlement ativo para C; **OU** role = **PROFESSOR** e `C.created_by == user.id`; **E**
3. Operação e estado do recurso estão autorizados. Na ausência de qualquer condição, **NEGAR**.

Regras de negócio: uma compra pendente, estornada ou revogada não libera acesso. Professor não obtém acesso automático a cursos de outros professores e, inicialmente, não acumula permissões de aluno. ADMIN pode alterar roles; jamais aceitar `role`, `is_admin` ou `purchased=true` informados pelo cliente como fonte de autoridade. O professor pode editar apenas cursos que produziu; publicação final fica reservada ao ADMIN na política inicial.

3.3 Erros de autorização

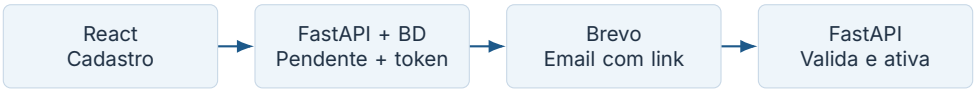
Sem credencial válida: **401**. Autenticado e proibido: **403**. Para recursos privados cuja existência deva ser ocultada, pode-se responder **404** de forma consistente. A UI pode esconder botões; a API deve impor todas as regras.

4 Cadastro e envio transacional de email — Brevo

Integração oficial: Brevo Transactional Email API. A Brevo **envia** emails de confirmação, recuperação e segurança, mas **não autentica**, não emite JWT e não define permissões. Somente o backend tem acesso à chave da API. O envio gratuito está sujeito às cotas e condições vigentes do plano (referência: **300 emails/-dia**, a confirmar no painel da Brevo).

4.1 Confirmação obrigatória do cadastro

1. `POST /auth/register`: validar dados, atribuir ALUNO e criar conta pendente, com hash Argon2id da senha. Estado inicial: `PENDING_VERIFICATION`.
2. Gerar token criptograficamente aleatório (**256 bits**), armazenar apenas seu **hash** e expiração **24 h**; publicar trabalho em fila de envio.
3. Worker chama a Brevo via `POST https://api.brevo.com/v3/smtp/email`; o template recebe o link de verificação e o destinatário.
4. React abre `/verificar-email?token=...` e encaminha o token para `POST /auth/verify-email`; o backend verifica hash, validade e uso único.
5. Marcar email verificado e conta `ACTIVE`, invalidar o token e liberar login. Reenvio cria novo token e invalida o anterior.



4.2 Configuração mínima e contrato

```
# Variaveis de ambiente do servico (nunca enviar ao React)
BREVO_API_KEY=<segredo_armazenado_em_secret_manager>
BREVO_SENDER_EMAIL=contato@dominio-verificado.com
BREVO_VERIFY_TEMPLATE_ID=<id_do_template>
PUBLIC_APP_URL=https://app.seudominio.com

POST https://api.brevo.com/v3/smtp/email
Headers: api-key: <BREVO_API_KEY>
       Content-Type: application/json
Body: {
 "to": [{"email": "aluno@exemplo.com"}],
 "templateId": 123,
 "params": {"VERIFY_URL": "https://app.seudominio.com/..."}
}
```

Operação: fila persistente com retentativas, idempotência, limite por usuário/IP e monitoramento de entregas, falhas e consumo da cota. Não registrar links contendo tokens, nem enviar senhas ou JWTs por email. Usar `Referrer-Policy: no-referrer` na página de confirmação. Recuperação de senha usa o mesmo provedor, token de uso único (**20 min**) e invalida sessões após troca.

Domínio: não é necessário para JWT, RBAC nem testes locais; para email real em produção, configurar remetente e domínio autenticado na Brevo (SPF, DKIM e política DMARC). Em desenvolvimento pode-se usar caixa de testes como Mailpit. Nunca montar o link de confirmação a partir de um cabeçalho `Host` não validado.

5 Tokens, sessões e segurança de login

5.1 JWT de acesso — obrigatório para usuários autenticados

O token é **assinado** com EdDSA (Ed25519), vence após **15 minutos** e carrega apenas dados necessários. JWT não é criptografado: **não incluir senha, email sensível, segredos ou informações financeiras**. APIs validam assinatura, algoritmo permitido, `iss`, `aud`, `exp`, `nbf`, `sub`, `sid`, status atual da conta e política de sessão.

```
{
 "sub": "<uuid-do-usuario>", "role": "ALUNO",
 "iss": "aprovaai-auth", "aud": "aprovaai-api",
 "typ": "access", "sid": "<uuid-da-sessao>",
 "jti": "<id-unico>", "iat": 1792780000,
 "nbf": 1792780000, "exp": 1792780900
}
```

Mudança de role: revogar sessões anteriores; se a aplicação usar claims de role, não confiar cegamente em roles antigas. Consultar role atual ou versão de autorização em armazenamento controlado. O esquema de chaves requer `kid`, guarda segura da chave privada e rotação controlada.

5.2 Refresh token, armazenamento e logout

Elemento	Política
Refresh token	Opaco, aleatório, 256 bits; validade de 7 dias; armazenar **hash** no BD; uso único e rotação atômica.
Reutilização	Se um token já rotacionado reaparecer, invalidar família de sessão e registrar alerta.
Access no React	Manter em memória, enviar `Authorization: Bearer <JWT>`; não persistir em `localStorage`.
Refresh no navegador	Cookie `HttpOnly`, `Secure` em produção, `SameSite` adequado, `Path` restrito; proteger o refresh contra CSRF.
Logout	Revogar refresh/sessão no servidor e limpar cookie; access JWT existente exige verificação de sessão/revogação para bloqueio imediato.
Múltiplos dispositivos	Sessões individuais identificadas por `sid`; suportar revogação individual e global.

5.3 Fluxos de login e recuperação

Login: `POST /auth/login` valida email/senha, conta `ACTIVE` e proteção contra tentativas excessivas. A role ADMIN utiliza o mesmo fluxo de autenticação por senha, mas suas rotas permanecem sujeitas às verificações de role e sessão ativa. Alteração de senha ou email exige autenticação recente. Recuperação de conta devolve resposta genérica para evitar enumeração.

5.4 Proteções de infraestrutura

Todas as rotas sensíveis usam HTTPS em produção, CORS restrito às origens aprovadas, CSRF nas operações por cookie, cabeçalhos seguros e `Cache-Control: no-store` em respostas privadas. Manter relógios sincronizados, rate limiting no login, cadastro, refresh e reenvio de emails, além de segredos fora do repositório e logs.

6 Autorização de cursos e modelo de dados

6.1 Direito de acesso do aluno e autoria do professor

ALUNO: um pagamento só concede acesso após confirmação idônea no servidor (webhook validado ou consulta confiável ao provedor). Persistir *entitlement* ou matrícula com `student_id`, `course_id`, `status`, início, validade e revogação. A verificação ocorre em *todas* as aulas, arquivos, provas privadas e streams, não só na página inicial do curso. **PROFESSOR:** em cada operação de edição ou leitura privada, exigir autoria `courses.created_by == user.id`; IDs arbitrários da URL não alteram o escopo.

Arquivos e vídeos. Nunca expor bucket público como única proteção. A API verifica a regra e fornece mídia via proxy autenticado ou URL assinada de curta validade. Links de prévia pública precisam estar marcados explicitamente. Negar acesso a compra estornada, vencida ou revogada.

6.2 Persistência PostgreSQL mínima

Tabela	Campos e condições principais
`users`	`id UUID PK`, `email UNIQUE`, `password_hash`, `role` (ALUNO/PROFESSOR/ADMIN), `status`, `verified_at`, `token_version`.
`courses`	`id UUID PK`, `created_by FK users`, `publication_status`, `title`.
`lessons`	`id UUID PK`, `course_id FK`, `is_preview`, `visibility`, referência segura da mídia.
`enrollments`	`student_id FK`, `course_id FK`, `status`, `valid_until`, `revoked_at`, vínculo com compra.
`sessions`	`id UUID`, `user_id FK`, `revoked_at`, datas de criação/última atividade.
`refresh_tokens`	`id`, `session_id FK`, `token_hash`, `expires_at`, `used_at`, `family_id`.
`email_tokens`	`user_id FK`, `token_hash`, `purpose`, `expires_at`, `consumed_at`.
`audit_events`	`actor_id`, `event`, `resource_id`, `outcome`, `occurred_at`, `request_id`.

6.3 Política de consistência e escopo

Uma role persistente por usuário simplifica a versão inicial. CONVIDADO não é linha de usuário. Apenas fluxo administrativo pode elevar ALUNO a PROFESSOR ou ADMIN; criar o primeiro ADMIN por mecanismo seguro de implantação. Contas `PENDING_VERIFICATION`, `SUSPENDED` ou `DISABLED` não mantêm acesso privado; suspensões revogam sessões. Regras financeiras e de autoria são validadas por consultas indexadas/transações consistentes; uma alteração deve refletir-se nas autorizações subsequentes.

Regra obrigatória de microsserviços: nem JWT válido nem a simples role `ALUNO` autorizam aula privada. A operação só é liberada após confirmação do *entitlement* para aquele `course_id`. A mesma lógica vale para autoria docente. O serviço que entrega o recurso é responsável pela decisão final, mesmo quando consulta um serviço de autorização central.

7 Contrato inicial da API e respostas

Os nomes de rotas são propostos. A documentação OpenAPI deve explicitar os esquemas de segurança, contratos de erro e direitos necessários para cada operação.

7.1 Identidade, sessões e Brevo

Método	Rota	Acesso / efeito
POST	`/auth/register`	Público; cria ALUNO pendente e agenda email Brevo.
POST	`/auth/verify-email`	Público; consome token válido e ativa conta.
POST	`/auth/resend-verification`	Público limitado; token novo e email via Brevo.
POST	`/auth/login`	Público; valida credenciais e conta ativa, inclusive para ADMIN.
POST	`/auth/refresh`	Cookie de refresh válido; rotação e JWT novo.
POST	`/auth/logout`	Sessão autenticada; revogação e limpeza de cookie.
POST	`/auth/forgot-password`	Público limitado; gera token e aciona Brevo; resposta genérica.
POST	`/auth/reset-password`	Token de uso único; troca hash e revoga sessões.
GET	`/auth/me`	Usuário autenticado; só dados do próprio perfil.
GET	`/auth/sessions`	Usuário autenticado; sessões próprias.
POST	`/auth/logout-all`	Usuário autenticado; revoga sessões da conta.

7.2 Cursos e administração

Método	Rota	Autorização
GET	`/courses`	Público; apenas cursos publicados.
GET	`/courses/{id}`	Público se publicado; não vazar conteúdo privado.
GET	`/courses/{id}/lessons/{id}`	ALUNO matriculado, PROFESSOR autor, ou ADMIN.
GET	`/courses/{id}/files/{id}`	Mesma verificação; gerar URL assinada curta.
POST	`/courses`	PROFESSOR ou ADMIN; atribuir autor pelo servidor.
PATCH	`/courses/{id}`	PROFESSOR autor ou ADMIN.
POST	`/admin/courses/{id}/publish`	Exclusivo ADMIN.
PATCH	`/admin/users/{id}/role`	Exclusivo ADMIN; auditar e revogar sessões.
GET	`/docs`, `/openapi.json`	Exclusivo ADMIN autenticado, com sessão de documentação válida.

7.3 Semântica de erros

400/422: entrada inválida; **401**: credencial ausente, inválida ou expirada; **403**: identidade válida sem autorização; **404**: recurso inexistente ou oculto por política; **409**: conflito de estado; **429**: limite de tentativas; **503**: dependência essencial indisponível. Usar `request_id`; não revelar se o email solicitado no fluxo de recuperação existe.

8 Swagger UI — acesso exclusivo do ADMIN

Exigência do produto: apenas a role ADMIN poderá obter a página Swagger, o esquema OpenAPI e qualquer rota alternativa de documentação. A restrição é **no backend**, não um simples botão oculto no React. Mesmo ADMIN precisa de conta ativa e sessão válida.

8.1 Duas barreiras que não podem ser confundidas

1. **Acesso à documentação:** navegação a `/docs` e busca de `/openapi.json` devem usar **cookie de sessão administrativa** `HttpOnly`, com JWT curto de finalidade `admin_docs` ou identificador de sessão opaco. O navegador normalmente não envia Bearer ao abrir uma página por URL; por isso a verificação administrativa precisa funcionar com a credencial de navegação.
2. **Uso das rotas dentro da documentação:** cada requisição de API disparada pelo Swagger continua exigindo `Authorization: Bearer <JWT>` e todas as verificações RBAC + curso. Acesso ao HTML **não** concede acesso automático às operações.

8.2 Esqueleto seguro no FastAPI

```
from fastapi import FastAPI, Depends
from fastapi.openapi.docs import get_swagger_ui_html
from fastapi.openapi.utils import get_openapi
from fastapi.responses import JSONResponse

app = FastAPI(docs_url=None, redoc_url=None,
          openapi_url=None)

async def require_admin_docs_session():
   # Implementar: cookie valido, sessao nao revogada,
   # usuario ACTIVE, role ADMIN atual.
   raise NotImplementedError("Implementar verificacao real")

@app.get("/openapi.json", include_in_schema=False)
async def openapi_admin(_=Depends(require_admin_docs_session)):
   return JSONResponse(get_openapi(
      title="AprovaAi API", version="1.0", routes=app.routes),
      headers={"Cache-Control": "no-store"})

@app.get("/docs", include_in_schema=False)
async def swagger_admin(_=Depends(require_admin_docs_session)):
   return get_swagger_ui_html(openapi_url="/openapi.json",
      title="AprovaAi | Swagger (ADMIN)",
      swagger_ui_parameters={"persistAuthorization": False})
```

ATENÇÃO: a dependência acima é *intencionalmente não implementada*; **não** deve ser publicada como proteção funcional. A implementação real verifica sessão, conta ativa e role atual a cada requisição. ReDoc deve ficar desabilitado ou protegido da mesma forma. Recursos estáticos da UI devem obedecer à política de segurança e CSP escolhida.

8.3 Jornada administrativa

ADMIN entra por `/admin/login` com email e senha, recebe cookie exclusivo de documentação, acessa `/docs` e, para testar endpoints, utiliza *Authorize* com access JWT administrativo de curta duração. Sessão expirada ou role removida bloqueiam imediatamente HTML, OpenAPI e APIs. Não usar *password grant* OAuth2 como novo fluxo moderno; se futuramente houver provedor OIDC, preferir *Authorization Code + PKCE*.

9 Operação, segurança e critérios de aceite

9.1 Proteções obrigatórias

- **Credenciais:** hashing Argon2id, política de tamanho mínimo e recuperação com tokens aleatórios de uso único.
- **Rede e navegador:** HTTPS; cookies `Secure`/`HttpOnly`; CSRF em fluxos com cookie; CORS restrito; validação de origem; proteção XSS/CSP.
- **Abusos:** rate limits por conta, IP e operação; reenvio Brevo limitado; respostas genéricas contra enumeração; alertas de login suspeito.
- **Segredos:** chave JWT privada e `BREVO_API_KEY` em gerenciador de segredos; nunca em JavaScript, Git ou logs; rotação planejada.
- **Auditoria:** registrar login, logout, falhas, alterações de role, revogações, acesso Swagger e negações relevantes com ator, alvo, resultado e `request_id`; nunca registrar tokens completos, senhas ou links de confirmação.

9.2 Checklist mínimo de testes automatizados

ID	Cenário / resultado exigido
T01	CONVIDADO visualiza apenas catálogo e prévias; acesso a aula privada negado.
T02	ALUNO com matrícula ativa acessa seu curso; sem matrícula ou com revogação é negado.
T03	PROFESSOR acessa curso de sua autoria; tentativa de acessar curso privado de outro autor é negada.
T04	ADMIN autenticado com sessão válida acessa todos os cursos e a Swagger UI.
T05	CONVIDADO, ALUNO e PROFESSOR não recebem `/docs`, `/openapi.json` ou ReDoc.
T06	JWT expirado, adulterado, com `iss`/`aud` errados ou sessão revogada é rejeitado.
T07	Refresh rotacionado não pode ser reutilizado; logout invalida a sessão prevista.
T08	Cadastro gera email via Brevo; token expira, é de uso único e não ativa duas vezes.
T09	Falha Brevo não ativa a conta; fila tenta novamente sem mensagens duplicadas indevidas.
T10	Payload com role ADMIN, autoria falsa ou compra forjada nunca concede acesso.

9.3 Ambientes e roteiro de implantação

Desenvolvimento: React em `localhost:5173`, API em `localhost:8000`, banco local e captura de email com Mailpit; não exige domínio. **Produção:** DNS controlado, HTTPS, cookies seguros, remetente Brevo autenticado (SPF/DKIM/DMARC), variáveis secretas e métricas de filas/cota. **Etapas:** (1) tabelas, (2) cadastro + Brevo, (3) login/JWT/refresh, (4) RBAC + cursos, (5) Swagger ADMIN, (6) testes e deploy.

Critério de conclusão: nenhuma operação privada é liberada apenas por conhecer URL ou identificador; nenhuma role comum acessa o Swagger; Brevo entrega emails mas a validação de tokens e as autorizações permanecem no backend.

Referências técnicas: FastAPI (segurança e OpenAPI), OWASP Authentication/Authorization/Session Management Cheat Sheets, RFC 7519 (JWT), RFC 9700 (OAuth2 Security BCP), documentação Brevo Transactional Email API. As cotas do plano gratuito podem mudar.